안녕하세요.
Javascript 강좌중인 Falcon 입니다.

오늘은 비교 연산자라는 것을 여러분들에게 알려드리려고 합니다. 개인적으로 Javascript를 처음으로 배우는 분들은 많지 않을거라고 생각합니다. Javascript는 다른 언어와는 달리 자료형이 없기 때문에 비교에 대해서도 차이점을 보입니다. 다른 언어에서 비교 연산자를 알고 있다고 해도 이 부분은 반드시 읽고 넘어가시길 바랍니다. 어려운 내용은 아닙니다.

```
> console.log(1 == 1);
  true                                  VM1916:1
<· undefined
> console.log(1 == 2);
  false                                 VM1920:1
```

프로그래밍 언어에서 ==은 같다라는 의미 입니다. 위는 1과 1을 비교하고 있으니 true가 나오게 되고, 1과 2를 비교하고 있으니 false가 나오게 됩니다. 이 부분은 다르지 않습니다.

```
> console.log("one" == "one");
  true                                  VM2013:1
<· undefined
> console.log("one" == "two");
  false                                 VM2019:1
```

당연히 문자열로 마찬가지로 문자열이 같으면 true가 문자열이 다르면 false가 나옵니다. 그런데 제가 말씀드렸다시피 변수 선언에는 var라는 키워드 하나밖에 없습니다. 숫자와 문자로된 숫자를 비교해보죠

```
> console.log(1 == '1');
  true                                  VM2075:1
<· undefined
> console.log(1 === '1');
  false                                 VM2100:1
<· undefined
```

위에서 첫 부분을 보면 숫자 1과 문자 1을 비교하는데 true가 나오게 됩니다. Javascript에서 ==은

같다라는 의미를 가지고 있지만 자료형에 따든 범위가 없습니다. 그래서 숫자1이든 문자 1이든 이를 1로 간주하여 true가 나오게 되는 겁니다. 이렇게 말하면 조금 이해가 잘되지 않을 겁니다.

```
> typeof 1
"number"
> typeof '1'
"string"
```

Javascript에서 자료형은 없고 var라는 키워드로 숫자, 문자, 실수, 문자열을 담을 수 있지만 해당하는 타입이 있습니다. 숫자는 number, 문자는 string입니다. ==는 이러한 타입을 전혀 보지 않기 때문에 true가 나오는 겁니다. 그래서 Javascript에서는 비교를 하기 위한 연산자가 하나 더 있습니다.

```
> console.log(1 === '1');
  false                                   VM2246:1
```

위에서 이미 보여드렸는데 바로 === 입니다. ===은 제가 위에서 보여드린 타입을 보기 때문에 숫자1은 숫자 1이여만 문자 1은 문자1 이여만 true가 나오게 됩니다. 이 부분을 잘기억해두시길 바랍니다.

```
> console.log(null == undefined);
  true                                    VM2517:1
<· undefined
> console.log(null === undefined);
  false                                   VM2525:1
<· undefined
```

이러한 경우가 몇가지가 더 있는데 이를 한 번 살펴보도록 하겠습니다. 먼저 null은 값이 없다라는 의미 입니다. 그리고 undefined는 값이 정의되지 않았다라는 의미 입니다.

```
> var num;
<· undefined
> console.log(num);
  undefined                               VM2604:1
<· undefined
```

위에서 빈 변수를 하나 선언을 하고 이를 출력을 해보면 num에 값이 존재하지 않기 때문에 undefined가 뜨게 됩니다. 즉, num에 아무런 값을 지정하지 않았다라는 의미 입니다.

```
> var num = null;
<· undefined
> console.log(num);
  null                                    VM2689:1
```

null은 값이 없다라는 뜻을 가지고 있습니다. 보통 무언가 만들 때 저도 자주사용하는 방법중의 하나입니다. 변수를 선언할 때 null을 넣어주고, console.log를 통해서 출력을 해보면 null이 뜨게 됩니다. 이는 여러분들이 선언을 해두고 이 변수를 사용하지 않았다라는 의미가 되고, 값이 없다라는 의미를 가지고 있습니다. 이 부분은 여러분들이 계속해서 공부를 하다보면 그렇구나하실 겁니다.

```
> console.log(null == undefined);
  true                                    VM2517:1
<· undefined
> console.log(null === undefined);
  false                                   VM2525:1
<· undefined
```

위의 경우도 사실 타입이 다르기 때문에 ==과 ===의 결과 값이 달라지니 알아두시기 바랍니다.

```
> console.log(true == 1);
  true                                    VM2974:1
<· undefined
> console.log(true === 1);
  false                                   VM2985:1
<· undefined
> console.log(true == '1');
  true                                    VM3019:1
<· undefined
> console.log(true === '1');
  false                                   VM3028:1
```

프로그래밍 언어에서는 숫자 1을 true로 생각을 합니다. 그렇기 때문에 ==으로 하면 타입을 보지 않기 때문에 true가 나오지만 ===으로 하게 되면 타입을 보기 때문에 이는 false가 됩니다.

```
> console.log(false == 0);
  true                                   VM3153:1
<· undefined
> console.log(false === 0);
  false                                  VM3158:1
<· undefined
> console.log(false == '0');
  true                                   VM3169:1
<· undefined
> console.log(false === '0');
  false                                  VM3180:1
<· undefined
```

위와 같은 내용 입니다. 프로그래밍 언어에서 false는 0으로 생각이 됩니다. ==과 ===의 차이를 확실하게 짚고 넘어가시기 바랍니다. 이건 조금 당연한 말이지만

```
> 0 == -0
<· true
> 0 === -0
<· true
```

0과 -0을 비교하면 무엇으로 하던 0이기 때문에 true가 나옵니다. 마지막으로 하나만 더 알고 넘어가죠.

```
> console.log(0/0);
  NaN                                    VM3294:1
<· undefined
> console.log(NaN == NaN);
  false                                  VM3371:1
<· undefined
> console.log(NaN === NaN);
  false                                  VM3390:1
```

NaN이라는 특수한 데이터 타입이 있는데 이는 0/0 이것은 수학으로 풀지 못하는 형태이죠. 이러한 것을 NaN형태로 나타내는데 이 두개는 풀지 못하는게 true가 나올 수 없기 때문에 false가 나오게 됩니다. 이 부분을 제외하고도 크다, 작다, 크거나같다, 작거나같다, 부정 등과 같은 다양한 비교연산자들이 있지만 이는 다든 프로그래밍 언어와 같습니다. 궁금하신 분들은 직접 찾아보시길 바랍니다.

JS Comparison Table
true false 1 0 -1 "true" "false" "1" "0" "-1" "" null undefined Infini···
dorey.github.io

위의 사이트는 ==과 ===에 대한 부분에 대해서 그림으로 그려져 있습니다.
윗 부분의 설명을 보면서 한 번 참고해보시기 바랍니다.

이것으로 [Javascript 강좌] 제 6강 비교 연산자 편을
마치도록 하겠습니다.